



Departamento de Computación y Sistemas Inteligentes

Ingeniería de Sistemas e Ingeniería Telemática

Arquitectura en la nube

Reto intermedio No. 1:

Construcción de un agente de IA que redimensiona imágenes en
AWS

Profesor: Gonzalo Llano R Ph. D

Santiago de Cali, mayo del 2026

Construcción de un agente de IA para procesamiento automático de imágenes en la nube con AWS

Introducción al laboratorio

En los últimos años, la evolución de los **modelos de lenguaje de gran escala (LLM)** ha permitido el desarrollo de una nueva generación de sistemas inteligentes conocidos como **agentes de inteligencia artificial**. Un agente de IA es un sistema capaz de **percibir información, razonar sobre una tarea y ejecutar acciones mediante herramientas externas**, permitiendo automatizar procesos complejos y tomar decisiones en entornos digitales.

A diferencia de los sistemas tradicionales de automatización, los agentes de IA pueden **interpretar instrucciones en lenguaje natural, seleccionar herramientas y ejecutar tareas en sistemas externos**, como bases de datos, servicios web o infraestructuras en la nube. Este paradigma está transformando la forma en que se construyen aplicaciones inteligentes, integrando **modelos de lenguaje, razonamiento automático y servicios computacionales**.

En el contexto de la computación en la nube, los agentes de IA permiten **orquestrar recursos y servicios cloud de manera inteligente**, facilitando la automatización de tareas como procesamiento de datos, análisis de información, administración de infraestructura y ejecución de pipelines de datos.

Los estudiantes deberán construir un **agente de IA básico utilizando un LLM y un framework de agentes**, el cual podrá **seleccionar y ejecutar servicios y recursos de TI para procesar imágenes almacenadas en AWS**.

El agente deberá analizar una solicitud del usuario y decidir utilizar una herramienta que desencadena una función **AWS Lambda**, la cual procesará imágenes almacenadas en **Amazon S3**.

Este laboratorio integra múltiples tecnologías utilizadas en sistemas modernos de inteligencia artificial y arquitecturas cloud, incluyendo almacenamiento de objetos con **Amazon S3**, procesamiento serverless mediante **AWS Lambda**, exposición de servicios mediante **Amazon API Gateway**, monitoreo mediante **Amazon CloudWatch**, y la construcción de agentes inteligentes mediante el framework **LangChain**.

A través de este laboratorio, los estudiantes aprenderán cómo **integrar modelos de lenguaje con herramientas externas para construir agentes capaces de ejecutar acciones en la infraestructura cloud**, comprendiendo los principios fundamentales del paradigma de **IA orientada a agentes**.

Objetivo general del laboratorio

Diseñar e implementar un **agente de IA básico capaz de seleccionar herramientas para ejecutar tareas en la nube**, utilizando un modelo de lenguaje (ChatGPT o Gemini), un framework de agentes ia, y servicios de AWS como **Amazon S3, AWS Lambda, API Gateway, y CloudWatch** para procesar imágenes automáticamente y monitorear en tiempo su ejecución. El agente de IA deberá:

- Interpretar instrucciones en lenguaje natural
- Decidir qué herramienta utilizar
- Ejecutar una acción en los servicios de AWS
- Modificar (redimensionar y rotar) imágenes almacenadas en S3
- Mostrar el resultado (imagen redimensionada y rotada)

Resultados de aprendizaje: al finalizar este laboratorio, el estudiante será capaz de:

- 1) Comprender el paradigma de agentes de inteligencia artificial
- 2) Diseñar arquitecturas en la nube para aplicaciones inteligentes
- 3) Implementar funciones *serverless* para procesamiento de datos
- 4) Construir agentes de IA capaces de utilizar herramientas externas
- 5) Integrar modelos de lenguaje con servicios y recursos en la nube
- 6) Validar y monitorear aplicaciones *serverless*

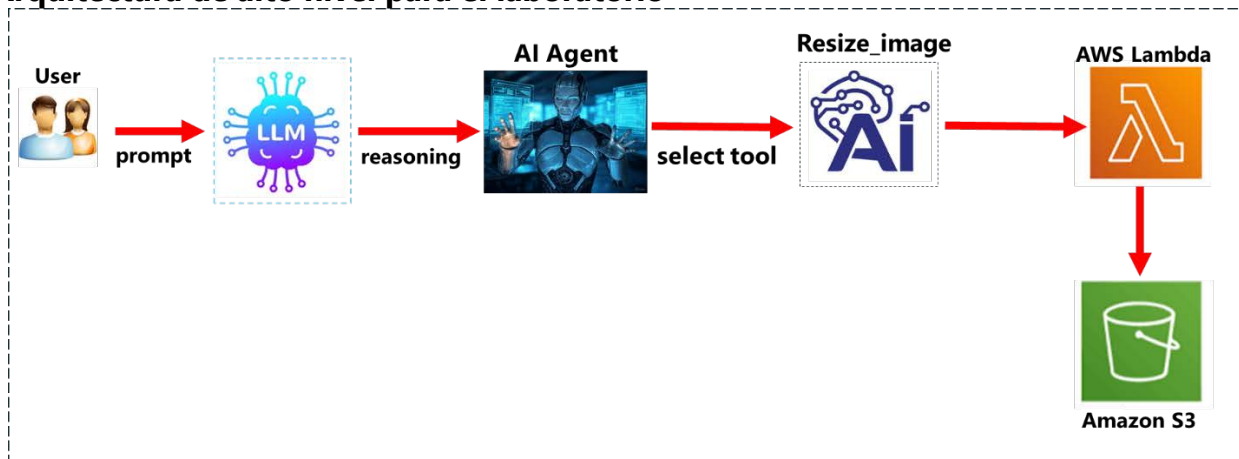
1. Tecnologías y herramientas a utilizar en el laboratorio

- **Servicios AWS:** Amazon S3, AWS Lambda, Amazon API Gateway, Amazon CloudWatch
 - **Framework para construir agentes de IA:** LangChain, LlamaIndex, CrewAI, OpenAI Function Calling
 - **Herramientas externas gratuitas:** Google Colab, ChatGPT, Google Gemini.
-

2. Flujo del laboratorio

Usuario → Prompt → LLM (ChatGPT/Gemini) → Agente IA (LangChain) → Selecciona Herramienta “resize_image” → Llama al API Gateway → Lambda procesa imagen → AWS S3 destino (imagen redimensionada)

3. Arquitectura de alto nivel para el laboratorio



4. Flujo del Sistema

- a) Subir una imagen muy grande (1280x1280) o superior al bucket de amazon **S3** (imagen original)
 - b) El usuario interactúa con el agente mediante un prompt. Ejemplo de un prompt: “**redimensionar la imagen original “imagen1.jpg” a un tamaño pequeño**”
 - c) El agente IA analiza la solicitud enviada en el prompt
 - d) El agente selecciona la herramienta: `resize_rotate_image_tool`
 - e) La herramienta invoca AWS Lambda
 - f) Lambda procesa la imagen
 - g) La imagen redimensionada y rotada se guarda en el bucket S3 destino
-

5. Componentes del agente IA:

A. LLM (Motor de razonamiento)

Puede ser: ChatGPT/ Google Gemini

Función:

- Interpretar el prompt del usuario
- Decidir qué herramienta usar

B. Agente Controller

Implementado con: `LangChain`

Función:

- Gestionar el flujo de razonamiento
- Seleccionar herramientas

C. Tools

Funciones que el agente puede ejecutar.

Ejemplo: `resize_image_tool`

Esta herramienta invoca AWS Lambda.

D. Infraestructura Cloud

Recursos utilizados: Amazon S3, AWS Lambda, Amazon API Gateway, Amazon CloudWatch.

6. Dependencias a utilizar en el proyecto (requirements.txt):

El laboratorio utiliza librerías de Python para integrar: modelos de lenguaje (LLM), agentes de IA, servicios cloud de AWS, procesamiento de imágenes, gestión de configuración.

```
- Langchain      # framework para desarrollar apps que utilizan LLM
- boto3          # SDK para interactuar con los servicios AWS desde python
- openai         # conectarse al LLM desde python
- python-dotenv  # gestión de variables de entorno
- pillow         # computación y procesamiento de imágenes
```

LangChain: Framework para desarrollar aplicaciones que utilizan LLM.

Función en el laboratorio: Construir el agente de IA y realizar las siguientes funciones:

- a) Interpretar el prompt del usuario
- b) Razonar sobre la tarea
- c) Decidir qué herramienta usar
- d) Ejecutar la herramienta

Ej.: El usuario escribe: "**redimensiona la imagen foto1.jpg a 768x768 y rota 90°**"

El agente usa LangChain para decidir: "Debo usar la herramienta `resize_rotate_image_tool`"

Ejemplo en el código:

```
from langchain.agents import initialize_agent
from langchain.agents import Tool
```

Concepto: Ingeniería de agentes de IA (AI Agents Engineering).

boto3: SDK que permite interactuar con servicios y recursos de AWS desde un programa escrito en Python.

- Configurar y desplegar buckets, recursos de computo (EC2, Dockers, Kubernetes)
- Subir y descargar archivos
- Ejecutar funciones Lambda
- Administrar bases de datos
- Administrar IAM

Función en el laboratorio: En este laboratorio Boto3 permite:

- Invocar la función **Lambda**
- Interactuar con los buckets **S3** (original y destino)
- Integrar el agente IA con los servicios y recurso de AWS

Ejemplo en el código

```
import boto3
lambda_client = boto3.client("lambda")
response = lambda_client.invoke(
    FunctionName="resize-rotate-image-function"
)
```

Concepto: Integración entre software y servicios de computación en la nube

openai: Esta librería permite conectarse a los LLM de OpenAI desde Python.

Función en el laboratorio: El agente utiliza el LLM como motor de razonamiento. El LLM interpreta el prompt y decide qué hacer.

Ejemplo en el código

```
from langchain.llms import OpenAI
llm = OpenAI(temperature=0)
```

Qué hace el LLM

- Convierte un prompt como: "**redimensiona la imagen foto1.jpg a 768x768 y rota 90°**"
- En una acción: "**Usar herramienta** resize_rotate_image_tool"

Concepto: Modelo LLM como motor de razonamiento del agente ia.

pillow: Librería de Python para **procesamiento de imágenes**. Permite en las imágenes: **a)** cambiar y/o modificar su tamaño **b)** rotar, **c)** recortar, **d)** convertir formatos

Función en el laboratorio: Se utiliza dentro de la **función Lambda** para redimensionar imágenes, rotarla y convertir formatos.

Ejemplo en el código:

```
from PIL import Image
image = Image.open(response["Body"])
resized = image.resize((width, height))
```

Concepto: Computacion y procesamiento de datos en la nube.

python-dotenv: permite cargar variables de entorno desde un archivo **xxx.env**.

Problema que resuelve: Evita colocar **credenciales dentro del código**.

Ejemplo incorrecto: `API_KEY = "123456789"`

Solución profesional: Usar archivo `xxx.env`.

```
OPENAI_API_KEY=123456789
AWS_REGION=us-east-1
```

Ejemplo en el código

```
from dotenv import load_dotenv
import os
load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")
```

Concepto: Gestión segura de configuración.

Cómo interactúan todas las dependencias

Usuario → prompt → LLM (OpenAI) → Agente (LangChain) → Tool(resize_image)→ API Gateway
→ AWS SDK (Boto3)→ AWS Lambda → Pillow → Imagen redimensionada en S3 destino

Tabla resumen de dependencias

Librería	Tipo	Función en el laboratorio
LangChain	Framework IA	Construcción del agente
Boto3	SDK AWS	Conectar Python con AWS
OpenAI	API LLM	Motor de razonamiento
Pillow	Procesamiento imagen	Redimensionar imágenes
python-dotenv	Configuración	Manejo de variables

6. Estructura de archivos final recomendada para el proyecto

```
ai-image-agent/
|
├─ agent.py          # lógica del agente
├─ tools.py          # herramientas a utilizar por el agente de ia
├─ config.py         # archivo de configuración
├─ requirements.txt  # instalación de dependencias
└─ README.md         # documentación técnica del proyecto
```

agent.py: código o lógica del agente IA

```
from langchain.agents import initialize_agent
from langchain.agents import Tool
from langchain.llms import OpenAI
from tools import resize_image_tool
llm = OpenAI(temperature=0)
tools = [
    Tool(
        name="ResizeImage",
        func=resize_image_tool,
        description="Redimensiona una imagen almacenada en un bucket S3"
    )
]
agent = initialize_agent(
```

```

tools,
llm,
agent="zero-shot-react-description",
verbose=True
)
prompt = input("Que tarea deseas realizar con la imagen? ")
response = agent.run(prompt)
print(response)

```

tools.py. Esta herramienta invoca la función Lambda.

```

import boto3
import json
lambda_client = boto3.client("lambda")
def resize_image_tool(bucket, image_name, width, height):
    payload = {
        "bucket": bucket,
        "image_name": image_name,
        "width": width,
        "height": height
    }
    response = lambda_client.invoke(
        FunctionName="resize-image-function",
        InvocationType="RequestResponse",
        Payload=json.dumps(payload)
    )
    result = json.loads(response["Payload"].read())
    return result

```

tools.py: herramienta invoca la función Lambda utilizada para redimensionar y rotar las imágenes. Esta función debe permitir:

- Procesar imágenes con múltiples formatos: **JPG, PNG, JPEG.**
- Generar 3 tamaños: Thumbnail(150x150), Medium(768x768), large (1024x1024)
- Rotar las imágenes de acuerdo a la solicitud del prompt
- Guardar las imágenes en carpetas distintas: thumbnails/; medium/; large/
- Revisar los logs almacenados en: **Amazon CloudWatch.**

Código básico de lambda. Para redimensionar y rotar las imágenes se debe importar la librería pillow. Deben modificar el código para que permita: **a)** varios tamaños de imagen, **b)** rotar las imágenes y **c)** acepte distintos tipos de formato (jpg, png, jpeg)

```

import json
from io import BytesIO
import boto3
from PIL import Image
import io

```

```

s3_client = boto3.client('s3')

def lambda_handler(event, context):
    # Obtener información del evento (bucket y archivo subido)
    bucket = event['Records'][0]['s3']['bucket']['name']
    key = event['Records'][0]['s3']['object']['key']

    # Descargar la imagen subida
    response = s3_client.get_object(Bucket=bucket, Key=key)
    image_content = response['Body'].read()

    # Abrir la imagen
    image = Image.open(BytesIO(image_content))

    # Redimensionar la imagen a un tamaño de thumbnail (ejemplo: 150x150)
    thumbnail_image = image.resize((150, 150))

    # Guardar el thumbnail en otro bucket o en el mismo con un nombre diferente
    buffer = BytesIO()
    thumbnail_image.save(buffer, 'JPEG')
    buffer.seek(0)

    # Subir el thumbnail de vuelta a S3
    s3_client.put_object(
        Bucket=bucket,
        Key=f'thumbnails/{key}',
        Body=buffer,
        ContentType='image/jpeg'
    )
    return {
        'statusCode': 200,
        'body': json.dumps('Imagen redimensionada con éxito')
    }

```

Archivo config.py: centraliza las **variables de configuración del agente y de AWS**, evitando que queden distribuidas en el código. Funciones principales:

- Configurar bucket S3 y la función Lambda
- Cargar credenciales desde variables de entorno
- Definir parámetros del agente ia

Código config.py

```

import os
from dotenv import load_dotenv

# Cargar variables desde archivo xxx.env
load_dotenv()

# -----
# CONFIGURACION SERVICIOS AWS
# -----
AWS_REGION = os.getenv("AWS_REGION", "us-east-1")

# Bucket donde se suben las imágenes
S3_BUCKET = os.getenv("S3_BUCKET", "ai-image-lab-bucket")

```



```

# Nombre de la función Lambda
LAMBDA_FUNCTION_NAME = os.getenv(
    "LAMBDA_FUNCTION_NAME",
    "resize-rotate-image-function"
)

# -----
# CONFIGURACION DEL AGENTE
# -----
MODEL_PROVIDER = os.getenv("MODEL_PROVIDER", "openai")
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

# Temperatura del modelo
LLM_TEMPERATURE = 0

# -----
# REDIMENSIONAR Y ROTAR LA IMAGEN
# -----
DEFAULT_WIDTH = 256
DEFAULT_HEIGHT = 256

```

Archivo `xxx.env`. (opcional)

```

OPENAI_API_KEY=tu_api_key
AWS_REGION=us-east-1
S3_BUCKET=ai-image-lab-bucket
LAMBDA_FUNCTION_NAME=resize-image-function
MODEL_PROVIDER=openai

```

NOTA: Este archivo evita colocar credenciales directamente en el código.

Archivo requirements.txt: Define las librerías necesarias para ejecutar el agente.

Código requirements.txt

```

langchain
openai
boto3
python-dotenv
pillow

```

Instalación: `pip install -r requirements.txt`

Archivo README.md: se utiliza en proyectos profesionales, explica: **a)** propósito del proyecto, **b)** cómo ejecutar el proyecto, **c)** flujo del proyecto

AI Image Processing Agent with AWS

Overview

This project implements a simple AI Agent capable of selecting tools to process images stored in AWS. The agent uses a Large Language Model (LLM) to reason about the user request and decide which tool to execute.

The selected tool invokes an AWS Lambda function that resizes images stored in Amazon S3. This project demonstrates how AI agents can interact with real cloud services.

Architecture

User Prompt → LLM (ChatGPT/Gemini) → AI Agent → Tool Selection → AWS Lambda → Amazon S3

Technologies Used

- Python
- LangChain
- AWS Lambda
- Amazon S3
- OpenAI API
- Boto3

Project Structure

```
ai-image-agent/  
agent.py  
tools.py  
config.py  
requirements.txt  
README.md
```

Prerequisites

Before running the project you need:

- Python 3.9+
- Access to AWS Academy Learner Lab
- An S3 bucket
- A deployed AWS Lambda function
- OpenAI API key (optional if using external LLM)

Installation

Clone the repository or download the files.

Install dependencies:

```
pip install -r requirements.txt
```

Environment Variables

Create a ``.env`` file with the following variables:

```
OPENAI_API_KEY=your_api_key  
AWS_REGION=us-east-1  
S3_BUCKET=ai-image-lab-bucket  
LAMBDA_FUNCTION_NAME=resize-image-function
```

Running the Agent

Run the agent: `python agent.py`

Example prompt:

Resize and rotate image foto1.jpg to 256x256 and 180° in bucket ai-image-lab-bucket

Expected Output

The resized image will appear in the S3 bucket.

Example:

```
foto1.jpg (original)
resized-foto1.jpg
rotated-foto1.jpg
```

Learning Objectives

Students will learn:

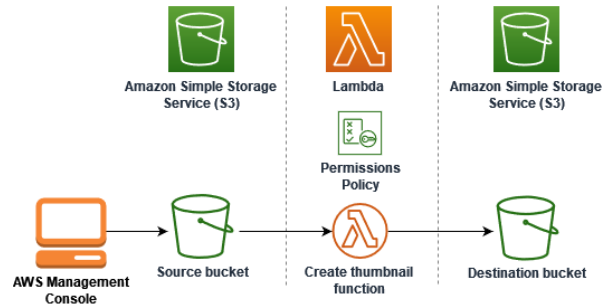
- How AI agents work
- How agents select tools
- Integration between AI and cloud services
- Serverless image processing
- Basic LangChain agent design

8. Prompt que deben utilizar para probar el agente

Los usuarios interactúan con el agente usando prompts.

- Ejemplo 1: Redimensionar y rotar 90°, 180°, 270°, la imagen foto1.jpg a 256x256 en el bucket estudiantes-imágenes
 - Ejemplo 2: Reduce la imagen paisaje.png a 128x128 y rota la imagen 180°
 - Ejemplo 3: Quiero crear una versión miniatura de la imagen selfie.jpg y rotar 270°
-

DESARROLLO DEL LABORATORIO



PASOS:

1) Configuración del bucket de S3:

- Configurar un bucket en **S3** donde los usuarios podrán subir las imágenes (objetos).
- Configurar el bucket para que cuando la imagen sea cargada, genere un evento que active la función Lambda.

Crear función:

Name: ImageResizeAgent
Runtime: Python 3.12
Role: LabRole

2) Configurar el disparador de S3:

- Configura un bucket de S3 para que envíe una notificación a la función Lambda cada vez que se suba un archivo.
 - Esto se hace desde la consola de S3, en la sección de **Properties -> Event Notifications**. Ahí puede crear un evento que dispare la función Lambda cuando un objeto es subido.
-

3) Crear el endpoint en la API Gateway

Ir a **Amazon API Gateway**

Crear API: ImageAgentAPI

Endpoint: POST /resize

Integración: Lambda → ImageResizeAgent

Desplegar: stage: prod

Guardar URL.